



Data & mémoire

AS M se M bleur

Plan

01 Pseudo-instructions

Retour sur certaines pseudo-instructions permettant de gérer la mémoire

02 Données immédiates

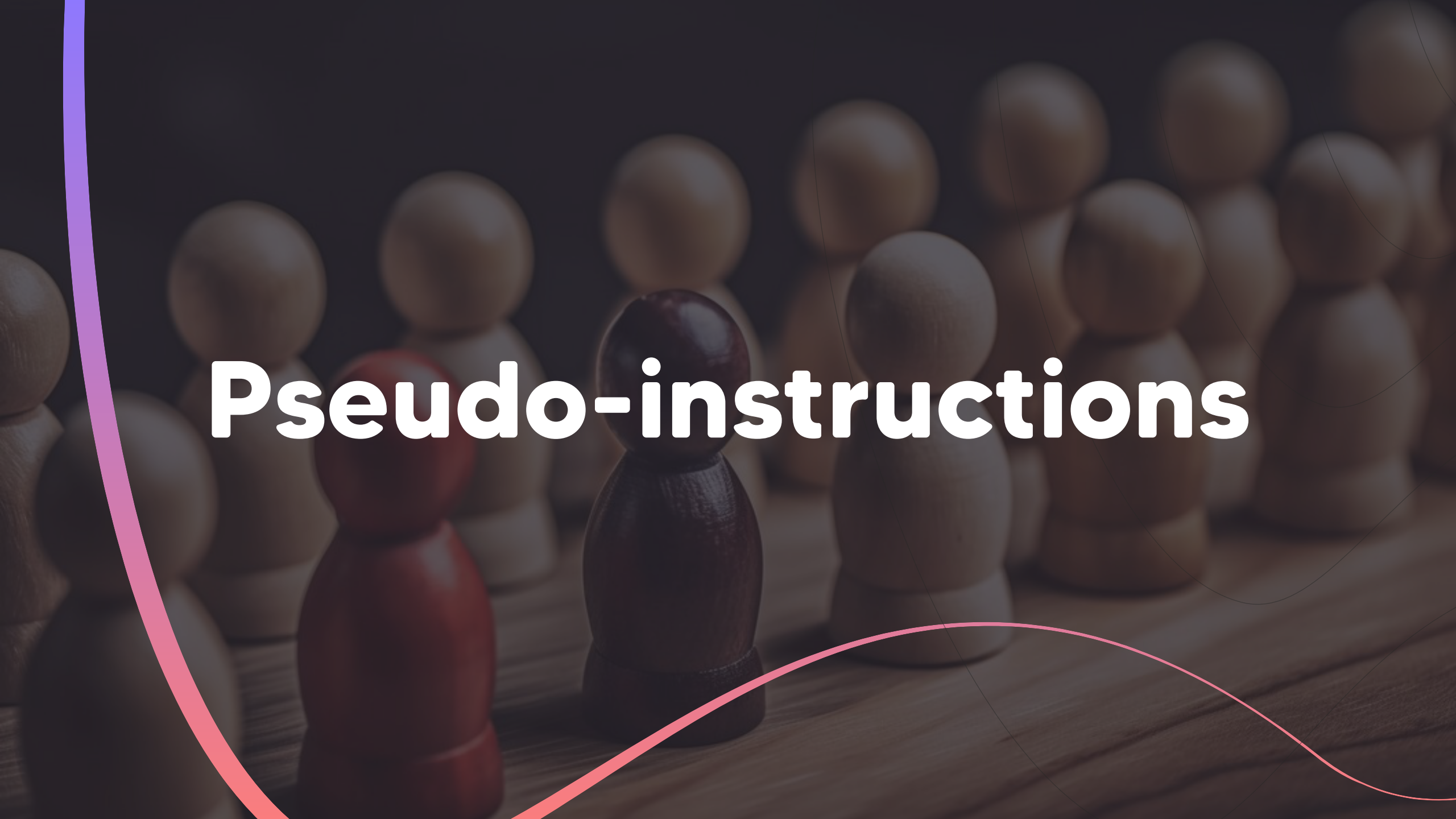
Quand les données sont directement dans l'instruction.

03 Indiens

Sont-ils petits ? Sont-ils grands ? Sont-ils sexuels ?

04 Modes d'adressages

Quand les données ne sont pas là où on les attends.

The background of the slide features a close-up, shallow depth-of-field photograph of several wooden chess pawns on a wooden board. The pawns are arranged in rows, with some in sharp focus and others blurred in the background. A dark, semi-transparent overlay covers the entire image. On the left side, there is a thick, curved line that transitions from purple at the top to pink at the bottom. Another thinner, curved pink line is visible in the lower right quadrant. The title 'Pseudo-instructions' is centered in a large, white, bold, sans-serif font.

Pseudo-instructions

Pseudo-instructions



Pas des vrais instructions !

Les pseudo-instructions ne sont pas traduites en vrai instructions du CPU.
Elles ne sont destinées qu'à l'assembleur.



Environnement

Un programme assembleur s'exécute dans un environnement : mémoire, data,

Les pseudo-instructions permettent de gérer cet environnement.



Mémoire

Les plus utilisées sont les pseudo-instructions qui gère les datas et leur place en mémoire.

EQU

Aucune mémoire n'est réservée.

Il s'agit juste d'un alias pour l'assembleur, d'un grand copier/remplacer dans tous le reste du texte.

DCD

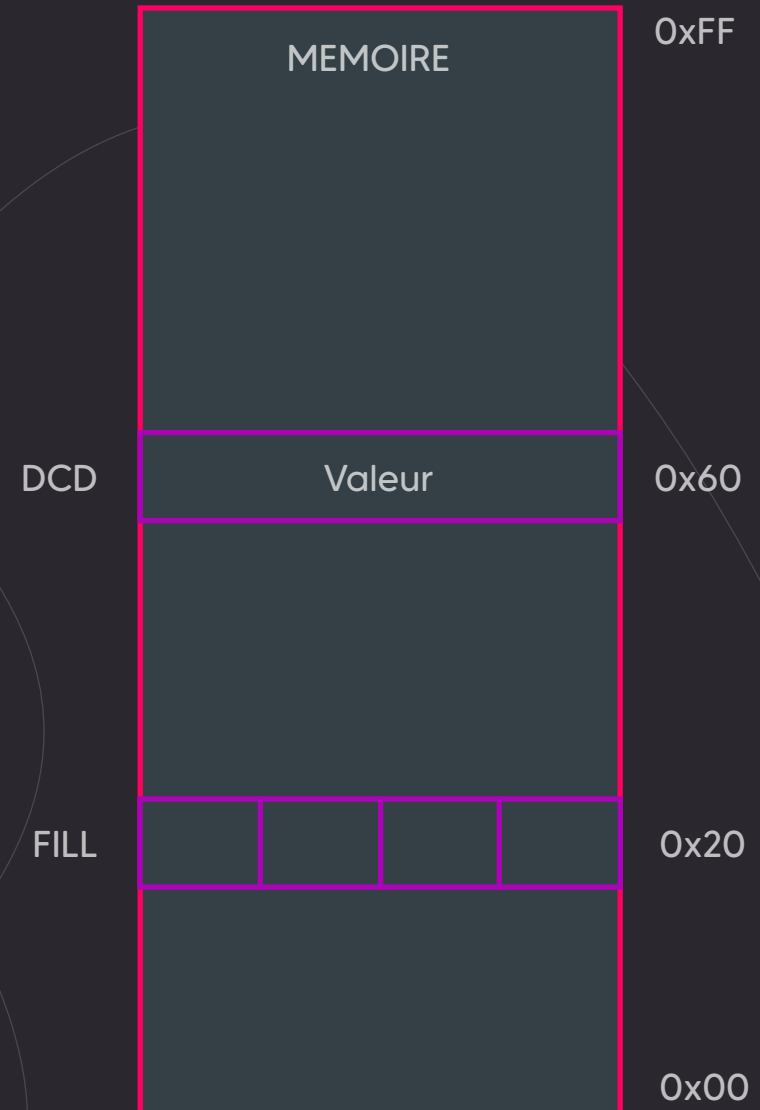
La mémoire est directement réservée par slot de 4 octets (un mot = 32 bits), et en plus elle est initialisé avec la valeur donnée en paramètre de DCD.

Le label en face de DCD correspond à l'adresse du début des 4 cases.

FILL

La mémoire est réservée octet par octet, mais elle n'est pas initialisé. La valeur passée à FILL correspond au nombre d'octets réservés.

Le label en face de FILL correspond à l'adresse de la première case réservée.



Données immédiates

A vintage-style stopwatch is centered in the background, slightly out of focus. It has a black casing and a white face with black markings. The main dial shows minutes from 0 to 15, and a sub-dial at the bottom shows seconds from 0 to 30. A pink wavy line starts from the left edge and curves across the bottom of the image. The overall background is dark and textured.

Données dans l'instruction

Data

Les données d'une instructions peuvent être dans un registre, en mémoire, ou directement passées à l'instruction

Inside

Si la donnée est directement passée à l'instruction, alors elle doit se trouver dans les 0 et les 1 décrivant l'instruction.

RISC

Une instruction RISC a toujours la même taille, dans notre cas 32 bits. Cette taille définit l'intégralité de l'instruction : l'opérateur, les paramètres, les données...

Taille ?

Si une instruction fait exactement 32 bits, et qu'on peut y mettre une data de 32 bits, comment faire ?

Example

00000000 00000000 00001011 10010000

#5620

5

MOV r0, #

00001010 00000000 1010 0101 10111001

decalage #5620

MOV r0, #5620

LDR =

Pseudo-instruction

L'assembleur remplace la pseudo-instruction par la valeur codée en dur dans le code et par une instruction qui charge cette valeur dans le registre.



LDR r0, =0xFF00FF00

Indiens



Mémoire = tableau

Grands indiens

La mémoire est vue comme un tableau, vu dans le sens de lecture : de gauche à droite, de haut en bas.

Pour stocker 0x12345678, on met donc 0x12 en haut à gauche, puis 0x34, puis 0x56 et on finit par 0x78.

0x12 (qui est l'octet de poids fort) est donc dans la première case (0).

C'est un stockage « Big Endian » (le poids fort en premier)

0x12345678

0 0x12	1 0x34	2 0x56	3 0x78
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19
20	21	22	23

Mémoire = immeuble

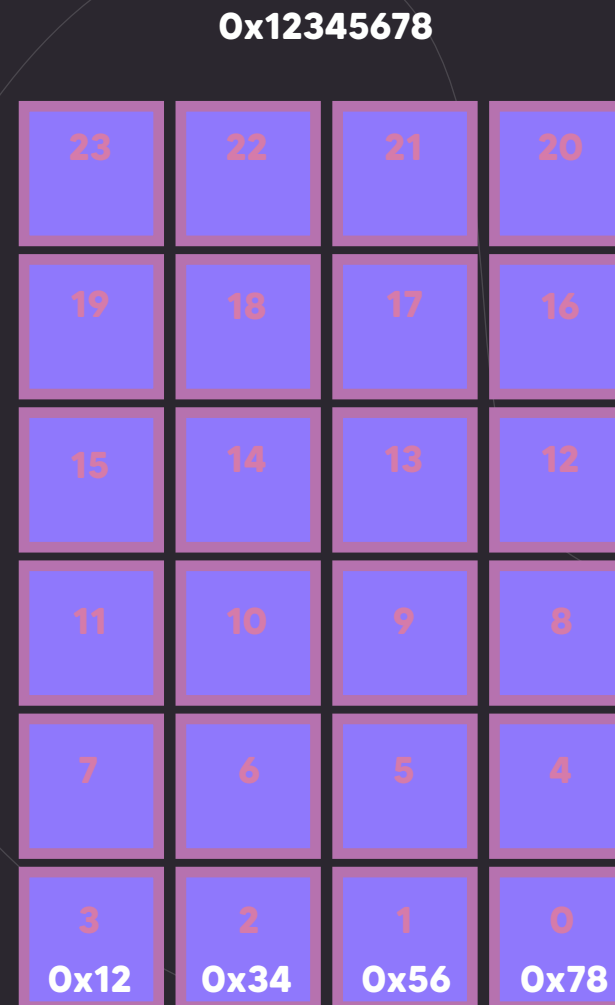
Petits indiens

La mémoire est vue comme un immeuble, avec les premiers étages en bas : la mémoire « basse » est la mémoire avec de petites valeurs, la mémoire « haute » avec de grandes valeurs.

Pour stocker 0x12345678, on met donc 0x12 en bas à droite, puis 0x34, puis 0x56 et on finit par 0x78.

0x78 (qui est l'octet de poids faible) est donc dans la première case (0).

C'est un stockage « Little Endian » (le poids faible en premier)

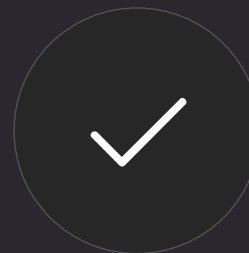


ARM ?



Différents besoins

Quel est le meilleur système d'adressage ?
Aucun, c'est juste une manière de voir la mémoire, il n'y a pas vraiment d'avantage à l'un ou à l'autre.



Différents standards

La plupart des CPUs sont maintenant Little-endian, mais c'était le contraire avant, du coup les protocoles réseaux (TCP/IP) sont en Big-endian.



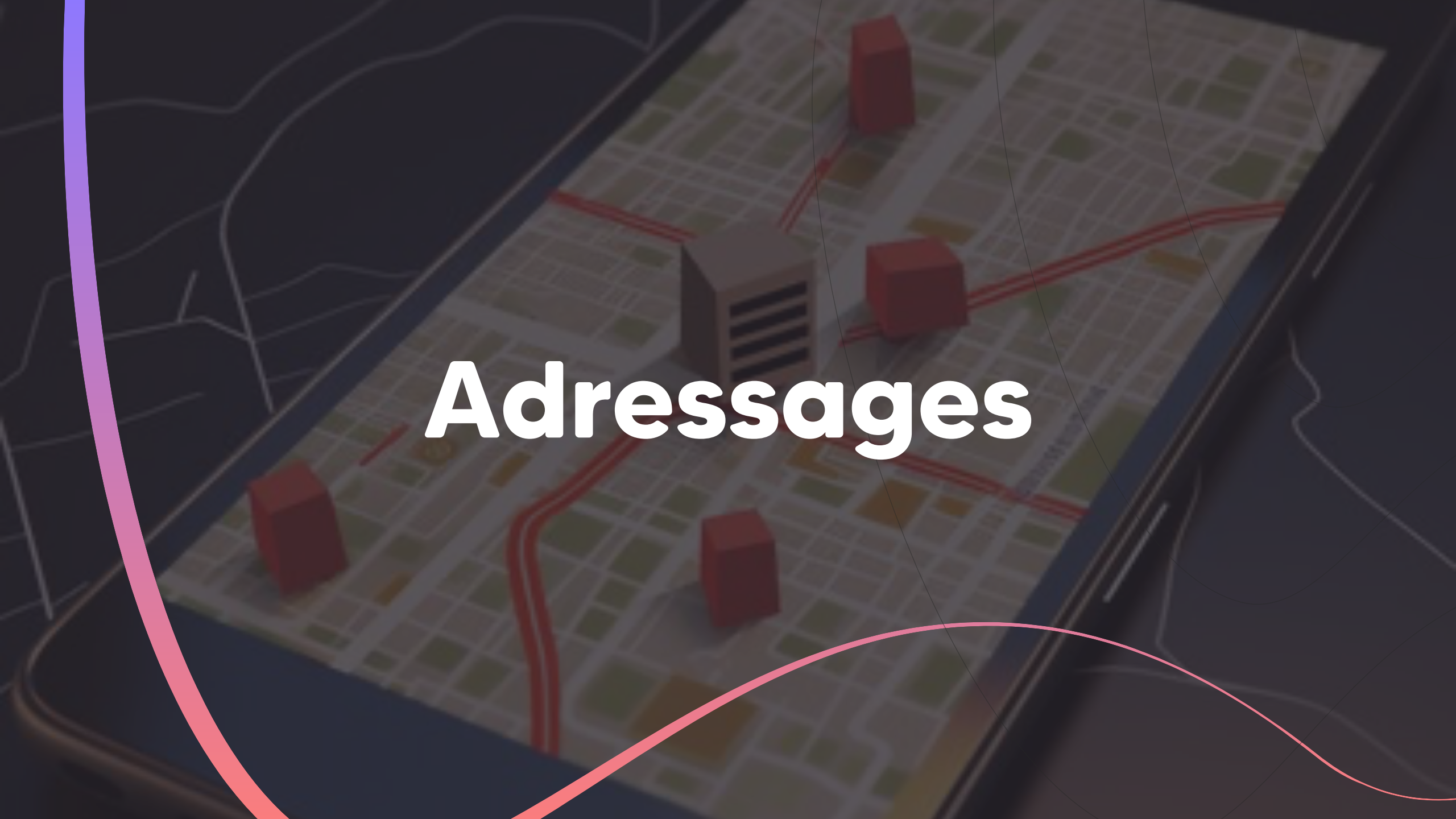
Bytesexual

ARM est « bytesexual » : il a un mode par défaut mais peut se configurer pour être soit l'un soit l'autre.



Default

Par défaut, ARM est comme les autres CPU : Little Endian.



Adressages



Direct

MOV rx, ry

La donnée est directement dans le registre, elle est directement accessible par le CPU.

Attention, à ne pas confondre avec
« immédiate » (encodée dans l'instruction)

Indirect

LDR rx, [ry]

La donnée n'est pas dans le registre, elle est dans une case mémoire pointée par un registre.

La valeur contenue dans le registre est interprétée comme une adresse.

Indirect indexé

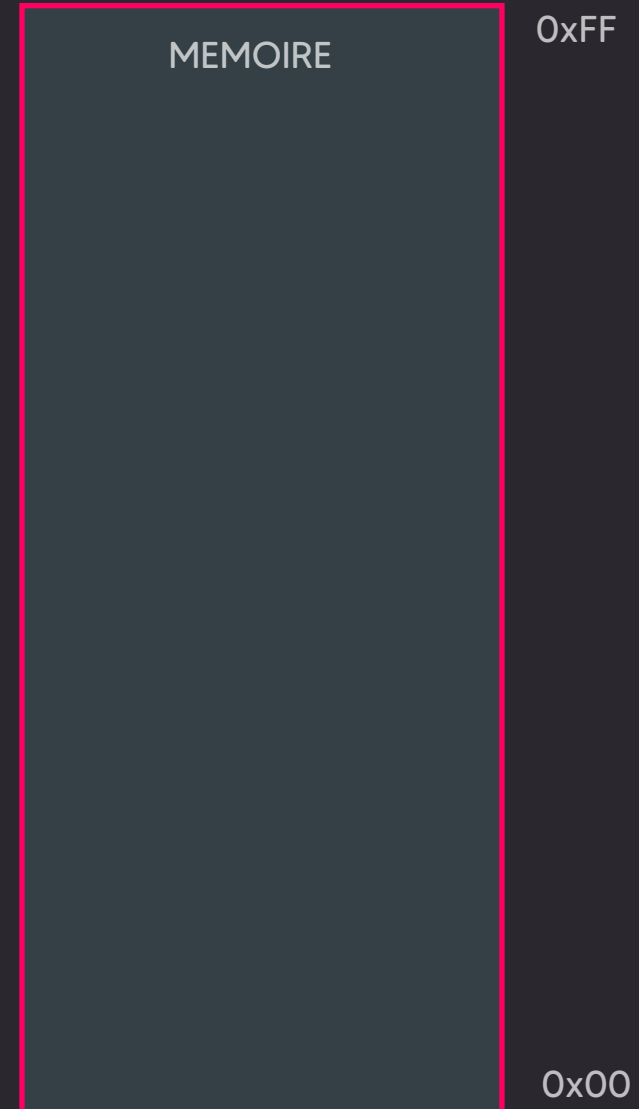
LDR rx, [ry, OFFSET]

La donnée n'est pas dans le registre, et la valeur dans le registre n'est pas directement l'adresse de la donnée, il faut ajouter un décalage.

Ce mode est utilisé pour accéder aux valeurs d'un tableau par exemple.

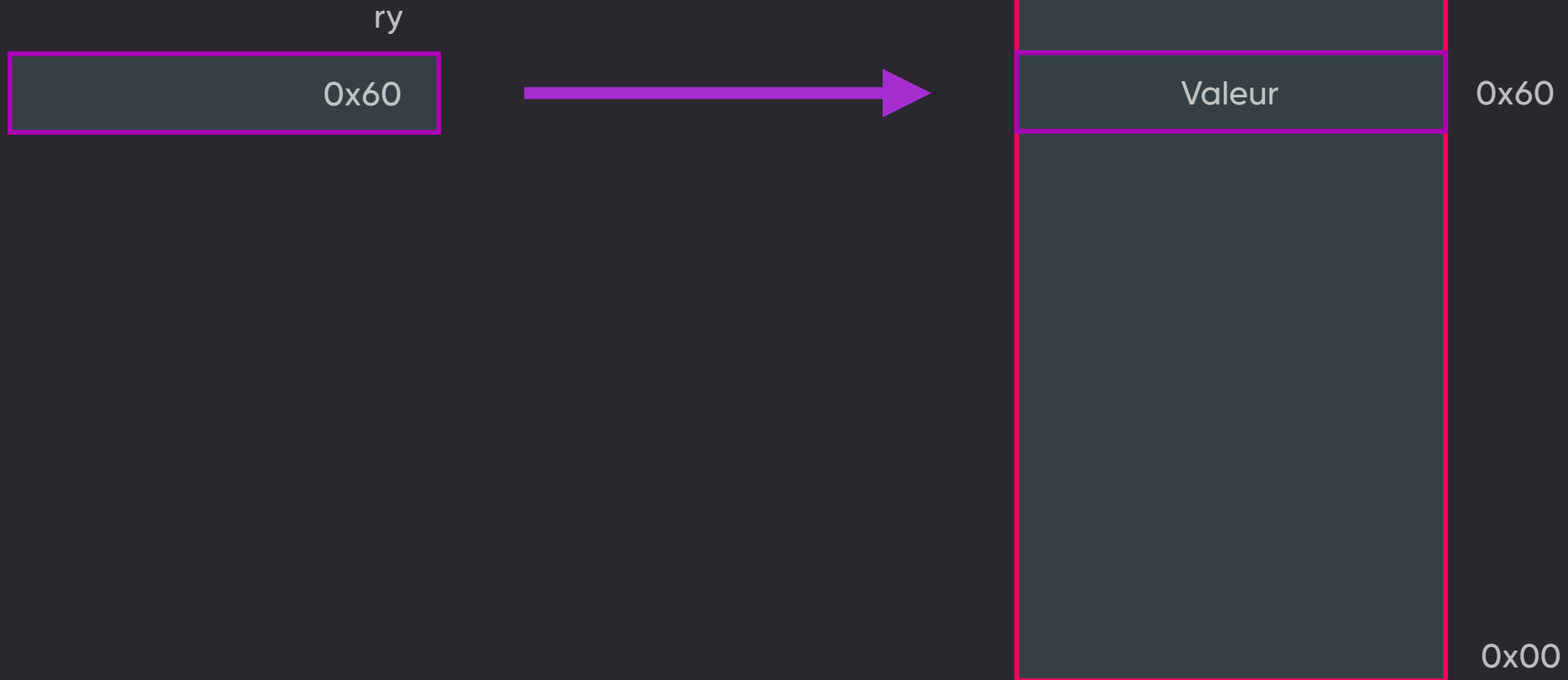
Adressage direct

MOV rx, ry



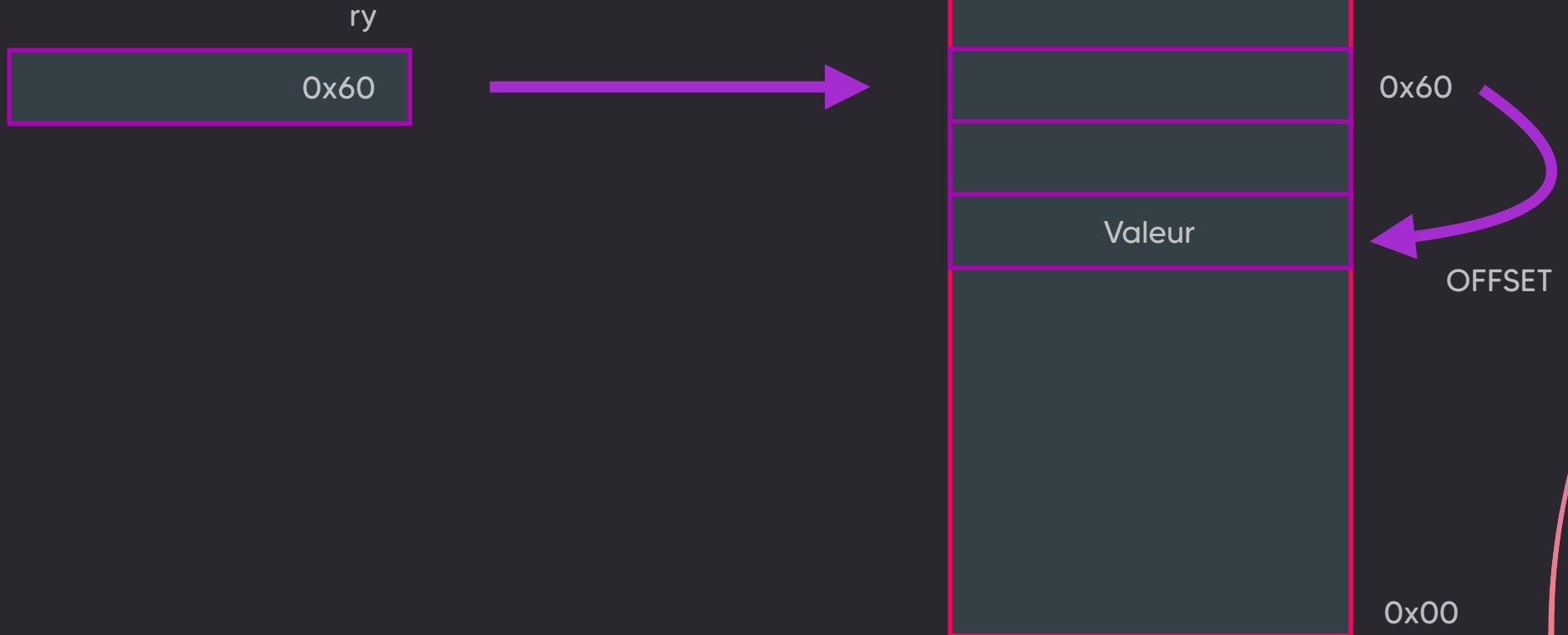
Adressage indirect

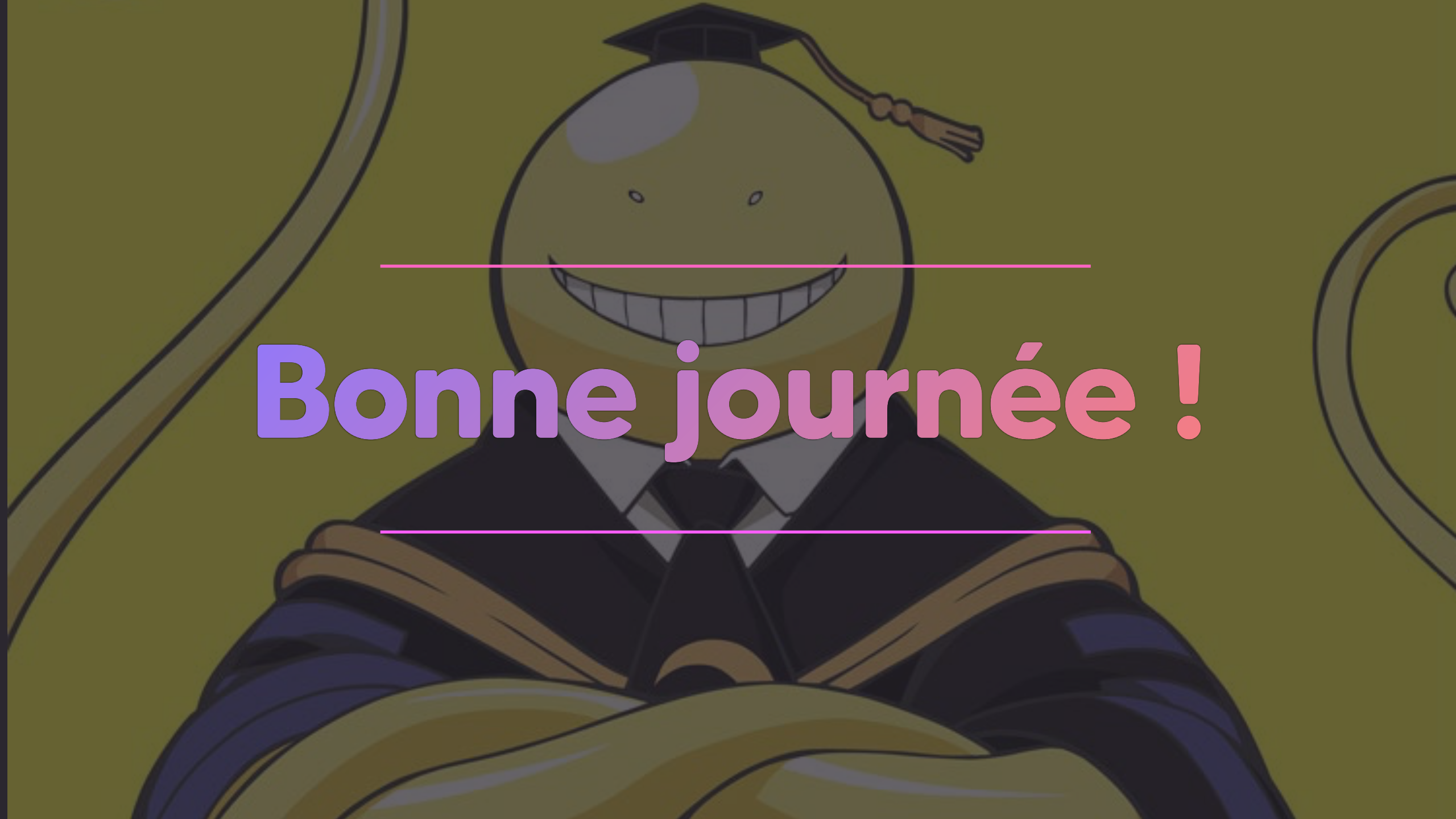
LDR rx, [ry]



Adressage indirect indexé

LDR rx, [ry, OFFSET]



A cartoon character with a round, light-colored head and a wide, toothy grin. The character is wearing a black graduation cap with a tassel and a black graduation gown with a blue stole. The background is a solid olive green color. Two horizontal pink lines are positioned above and below the text.

Bonne journée !